

Correction des exercices sur les calculs inductifs

Enseignement à distance de l'Université Marie et Louis Pasteur
Spécification et preuve de programmes

Version du 9 décembre 2025

1 Table de vérité

Définition d'une fonction WhyML qui implémente le “ou” logique conformément à sa table de vérité :

```
let function or (x y : bool) : bool
= match x with
| False →
  match y with
  | False → False
  | True → True
  end
| True →
  match y with
  | False → True
  | True → True
  end
end
end
```

2 Type inductif des entiers naturels

On adopte la définition inductive suivante du type $\mathbb{N}$ des entiers naturels.

1. 0 est un entier naturel.
2. Si n est un entier naturel, alors $S(n)$ est un entier naturel.
3. L'ensemble $\mathbb{N}$ est le plus petit ensemble qui contient les entiers naturels définis par les règles 1 et 2.

Remarque importante : Pour éviter tout risque de confusion avec l'addition, qu'on définit dans cet exercice, l'entier naturel qui suit immédiatement l'entier naturel n dans l'ordre croissant des entiers n'est pas noté $n + 1$, mais $S(n)$, qui se lit “successeur (immédiat) de n ”.

1. $plus : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$.
2. La fonction $plus$ qui réalise dans $\mathbb{N}$ l'addition de deux entiers naturels est définie par les égalités suivantes :

$$plus(x, 0) = x \tag{1}$$

$$plus(x, S(i)) = S(plus(x, i)) \tag{2}$$

- Après avoir défini un type WhyML pour les entiers naturels, on peut coder cette fonction en WhyML, comme suit :

```
type nat = Z | S nat

let rec function plus (x y : nat) : nat
= match y with
  | Z → x
  | S i → S (plus x i)
end
```

- La fonction $mult : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$ qui réalise le produit de deux entiers naturels de $\mathbb{N}$ est définie par les égalités suivantes :

$$mult(x, 0) = 0 \quad (3)$$

$$mult(x, S(i)) = plus(mult(x, i), x) \quad (4)$$

- Voici un code WhyML pour cette fonction :

```
let rec function mult (x y : nat) : nat
= match y with
  | Z → Z
  | S i → plus (mult x i) x
end
```

- La copie de ce code WhyML complet dans l'interface en ligne de Why3, suivie par un clic sur l'icône  qui représente une baguette magique ne donne pas d'erreur, donc ce code est syntaxiquement correct et le typage est respecté.

3 Fonction miroir

Le fonction `reverse` du fichier <https://www.why3.org/stdlib/list.html> de la bibliothèque standard de Why3 définit une fonction WhyML qui implémente la fonction *miroir* du cours.

4 Calcul de sommes

- $somme-liste : Liste(\mathbb{N}) \rightarrow \mathbb{N}$
-

$$\begin{aligned} somme-liste(Nil) &= 0 \\ somme-liste(Cons(x, l)) &= x + somme-liste(l) \end{aligned}$$

- Ces égalités peuvent être implémentées en WhyML par la définition suivante d'une fonction `sommeListe`.

```
use int.Int
use list.List
```

```

let rec function sommeListe (l : list int) : int
= match l with
| Nil → 0
| Cons x l → x + sommeListe l
end

```

4. Type : $liste-sommes : Liste(Liste(\mathbb{N})) \rightarrow Liste(\mathbb{N})$.

Égalités :

$$\begin{aligned}
liste-sommes(Nil) &= Nil \\
liste-sommes(Cons(l, r)) &= Cons(somme-liste(l), liste-sommes(r))
\end{aligned}$$

5. Ces égalités peuvent être implémentées en WhyML par la définition suivante d'une fonction `listeSommes`.

```

let rec function listeSommes (l : list (list int)) : list int
= match l with
| Nil → Nil
| Cons l r → Cons (sommeListe l) (listeSommes r)
end

```

6. Type : $aplatir : Liste(Liste(\alpha)) \rightarrow Liste(\alpha)$.

Égalités :

$$\begin{aligned}
aplatir(Nil) &= Nil \\
aplatir(Cons(l, r)) &= append(l, aplatir(r))
\end{aligned}$$

7. Voici une implémentation de cette fonction en WhyML, qui utilise la concaténation `++` définie dans le module `Append` du fichier `list.mlw` de la bibliothèque standard de Why3.

```

use list.Append

let rec function aplatir (l : list (list 'a)) : list 'a
= match l with
| Nil → Nil
| Cons l r → l ++ aplatir r
end

```